ArgOS v2: World-Building Architecture

> **Companion to ARCHITECTURE.md** - This document covers how GodAI builds simulations. ARCHITECTURE.md covers how individual agents think.

1. Overview

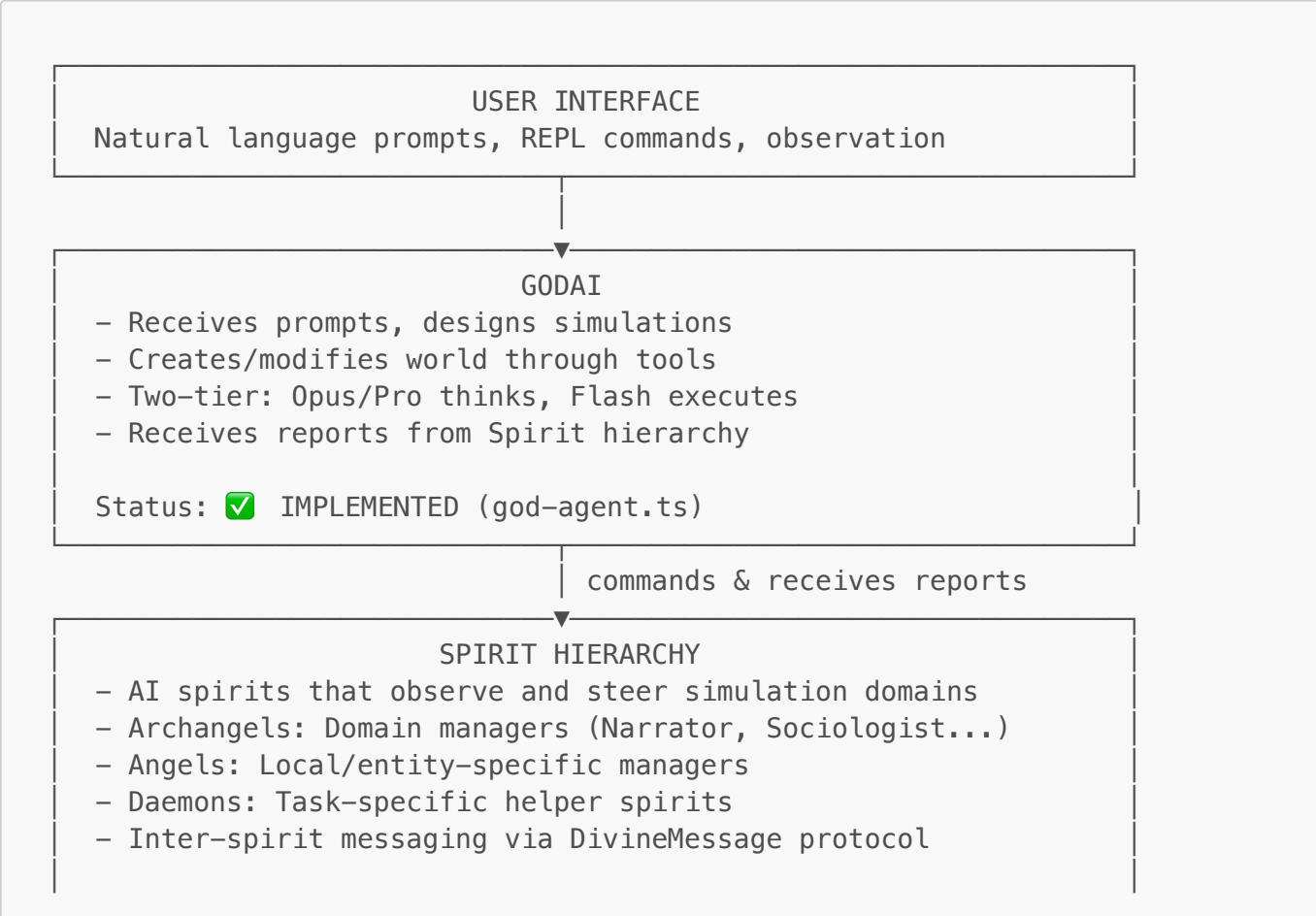
ArgOS is a **Linguistic Simulation Engine** - a platform where AI agents autonomously build and run simulations using an Entity-Component-System (ECS) architecture. The key insight is that the ECS serves as a **programmable substrate** that a "GodAI" can read from and write to, creating emergent simulations from natural language descriptions.

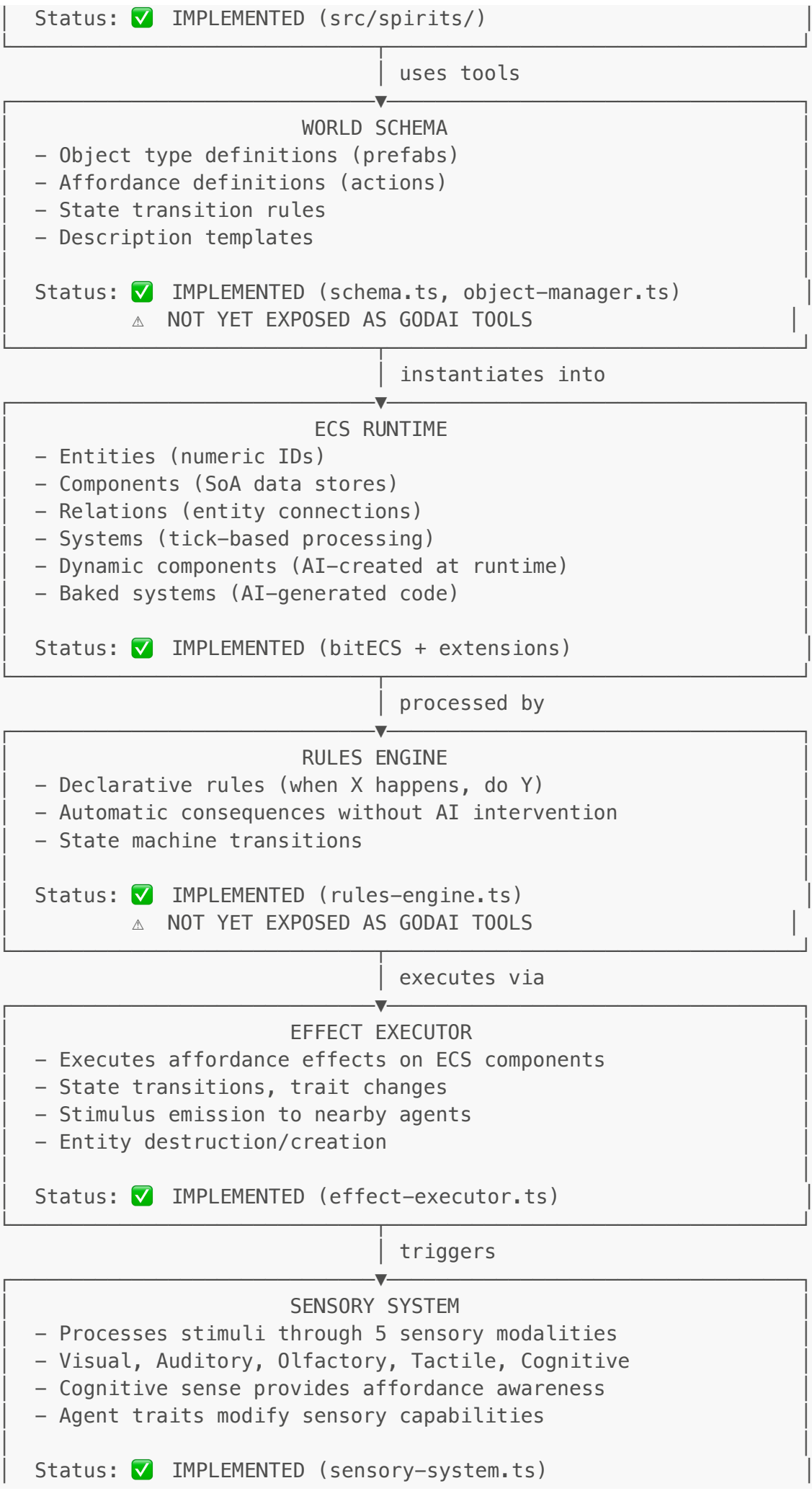
Core Principles

1. **AI-Driven World Building** - GodAI creates components, entities, systems, and rules from natural language prompts
2. **ECS as Execution Layer** - BitECS provides the deterministic, high-performance runtime
3. **Schema as Contract** - WorldSchema defines templates that bridge AI creativity and structured execution
4. **Text-First Perception** - Agents perceive the world through MUD-style text rendering
5. **Emergent Behavior** - Rules and systems create consequences without explicit AI micromanagement

---

2. Architecture Layers




Status: ✅ IMPLEMENTED (src/spirits/)
uses tools
WORLD SCHEMA
- Object type definitions (prefabs)
- Affordance definitions (actions)
- State transition rules
- Description templates
Status: ✅ IMPLEMENTED (schema.ts, object-manager.ts)
⚠ NOT YET EXPOSED AS GODAI TOOLS
instantiates into
ECS RUNTIME
- Entities (numeric IDs)
- Components (SoA data stores)
- Relations (entity connections)
- Systems (tick-based processing)
- Dynamic components (AI-created at runtime)
- Baked systems (AI-generated code)
Status: ✅ IMPLEMENTED (bitECS + extensions)
processed by
RULES ENGINE
- Declarative rules (when X happens, do Y)
- Automatic consequences without AI intervention
- State machine transitions
Status: ✅ IMPLEMENTED (rules-engine.ts)
⚠ NOT YET EXPOSED AS GODAI TOOLS
executes via
EFFECT EXECUTOR
- Executes affordance effects on ECS components
- State transitions, trait changes
- Stimulus emission to nearby agents
- Entity destruction/creation
Status: ✅ IMPLEMENTED (effect-executor.ts)
triggers
SENSORY SYSTEM
- Processes stimuli through 5 sensory modalities
- Visual, Auditory, Olfactory, Tactile, Cognitive
- Cognitive sense provides affordance awareness
- Agent traits modify sensory capabilities
Status: ✅ IMPLEMENTED (sensory-system.ts)

```
                                    │
                                    │ rendered to
                                    ▼
┌─────────────────────────────────────────────────────────────────┐
│                        OUTPUT LAYER                             │
│  - TextRenderer: MUD-style descriptions for agent perception    │
│  - Graphics: Pixi.js 2D visualization (optional)                │
│  - ASCII World: Grid-based tile maps                            │
│                                                                 │
│  Status: ✅ IMPLEMENTED (text-renderer.ts)                      │
│          ✅ WIRED TO AGENT COGNITION (sensory-system.ts)        │
└─────────────────────────────────────────────────────────────────┘
```

---

3. GodAI Capabilities

3.1 Current Tools (Implemented)

Tool	Purpose	Status
`createAgent`	Create cognitive NPC with LLM thinking	✅
`createEntity`	Create mechanical entity (no cognition)	✅
`createRoom`	Create location/space	✅
`createObject`	Create physical object	✅
`createStimulusSource`	Create periodic event emitter	✅
`createComponent`	Define new component type at runtime	✅
`setDynamicComponent`	Attach dynamic component to entity	✅
`setComponentValues`	Modify component data	✅
`addRelation`	Create relationship between entities	✅
`bakeNewSystem`	Generate new system from description	✅
`activateSystem` / `deactivateSystem`	Control system execution	✅

Spirit Management Tools - See **Section 13.7** for the 7 spirit hierarchy tools (`getSpiritHierarchy`, `getSpiritReports`, `sendDirectiveToSpirit`, etc.)

3.2 Planned Tools (To Integrate)

Tool	Purpose	Source
`spawn`	Instantiate from WorldSchema prefab	ObjectManager
`defineObjectType`	Add new prefab to schema	WorldSchema
`defineAffordance`	Add new action type	WorldSchema
`defineRule`	Add declarative rule	RulesEngine

Tool	Purpose	Source
`describeEntity`	Set/update entity description	TextRenderer

3.3 Two-Tier Execution Model

```
User Prompt: "Create a medieval village with trading NPCs"
                    │
                    ▼
┌─────────────────────────────────────────────────────────┐
│  PLANNER (Opus/Pro) - "Thinking" Phase                  │
│                                                         │
│  1. Analyzes prompt requirements                        │
│  2. Designs component schema                            │
│  3. Plans entity structure                              │
│  4. Designs systems and rules                           │
│  5. Outputs structured DesignDocument                   │
│                                                         │
│  Model: gemini-2.5-pro with extended thinking           │
└─────────────────────────────────────────────────────────┘
                              │ DesignDocument
                              ▼
┌─────────────────────────────────────────────────────────┐
│  EXECUTOR (Flash) - "Building" Phase                    │
│                                                         │
│  1. Receives DesignDocument                             │
│  2. Calls tools to create components                    │
│  3. Spawns entities                                     │
│  4. Bakes systems                                       │
│  5. Sets up initial state                               │
│                                                         │
│  Model: gemini-2.0-flash (fast, cheap)                  │
└─────────────────────────────────────────────────────────┘
```

---

4. WorldSchema Layer

4.1 What WorldSchema Provides

WorldSchema is a **registry of templates** that sits between GodAI and the raw ECS. It provides:

- **Object Type Definitions (Prefabs)** - Templates for common objects
- **Affordance Definitions** - What actions can be performed
- **State Machines** - How objects transition between states
- **Description Templates** - How objects are described in text

```
// Object Type Definition (Prefab)
{
  type: "torch",
```

```
  components: ["Position", "Renderable", "Perceivable"],
  traits: ["lightSource", "takeable", "examinable"],
  defaultState: "lit",
  states: {
    "lit": {
      description: "A burning torch casting flickering light",
      emits: { light: 1.0 }
    },
    "unlit": {
      description: "An unlit torch",
      emits: { light: 0 }
    }
  }
}

// Affordance Definition (Action)
{
  name: "extinguish",
  requires: ["lightSource"],
  blockedBy: ["unlit"],
  transitions: { "lit": "unlit" },
  descriptionTemplate: "{actor} extinguishes {target.name}"
}
```

4.2 Current vs Target Flow

Current Flow (Disconnected):

```
GodAI → createObject tool → raw ECS entity (no affordances)
         WorldSchema/ObjectManager exist but GodAI doesn't use them
```

Target Flow (Integrated):

```
GodAI → spawn("torch") → WorldSchema lookup → ECS entity with:
                                               - All components
                                               - Traits attached
                                               - State machine ready
                                               - Affordances available
                                               - Description template
```

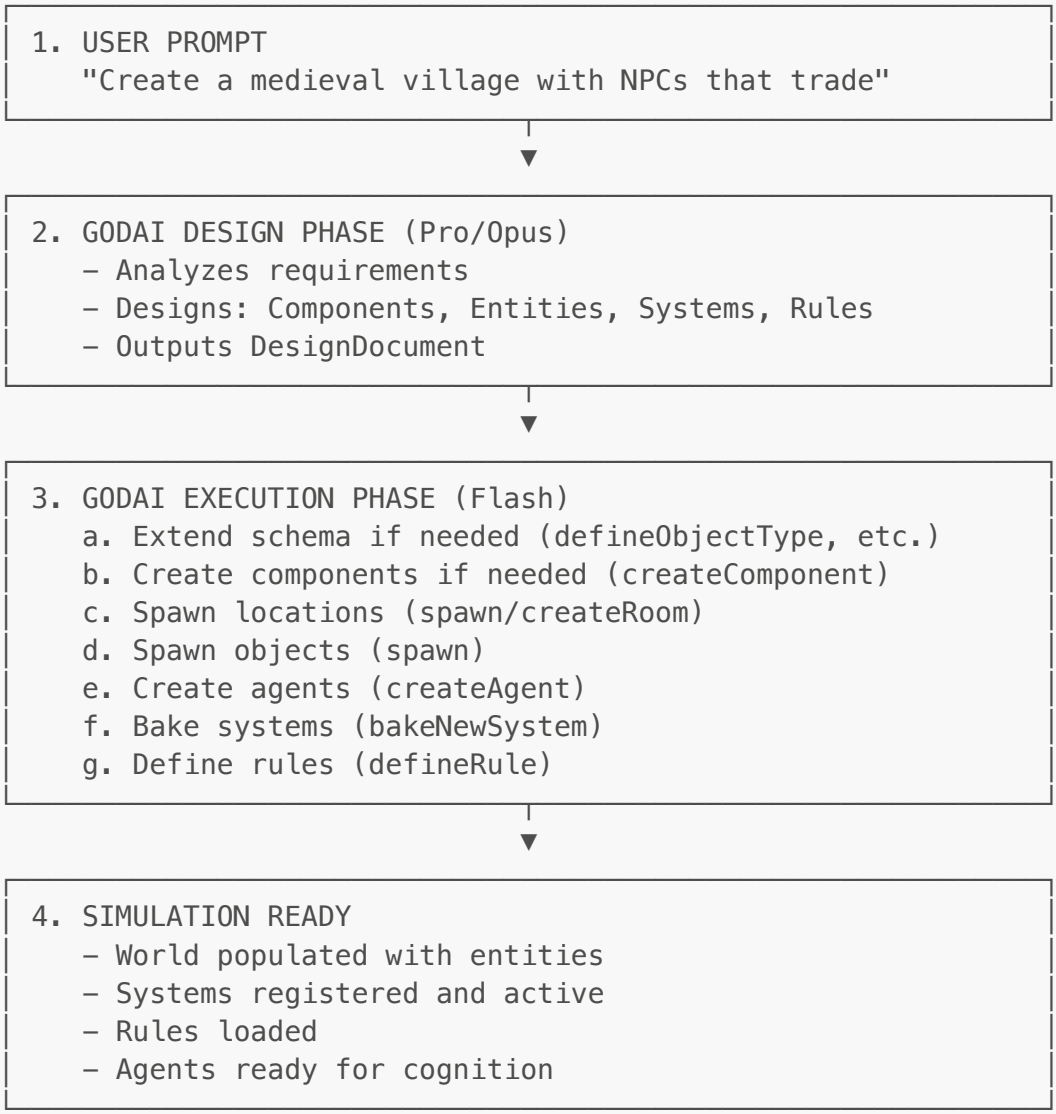
4.3 Integration Requirements

1. **Add `spawn` tool to GodAI** - Wraps ObjectManager.spawn()
2. **Add `defineObjectType` tool** - Extends WorldSchema at runtime
3. **Add `defineAffordance` tool** - Adds new actions to schema
4. **Wire TextRenderer to cognition** - Agents perceive via rendered text

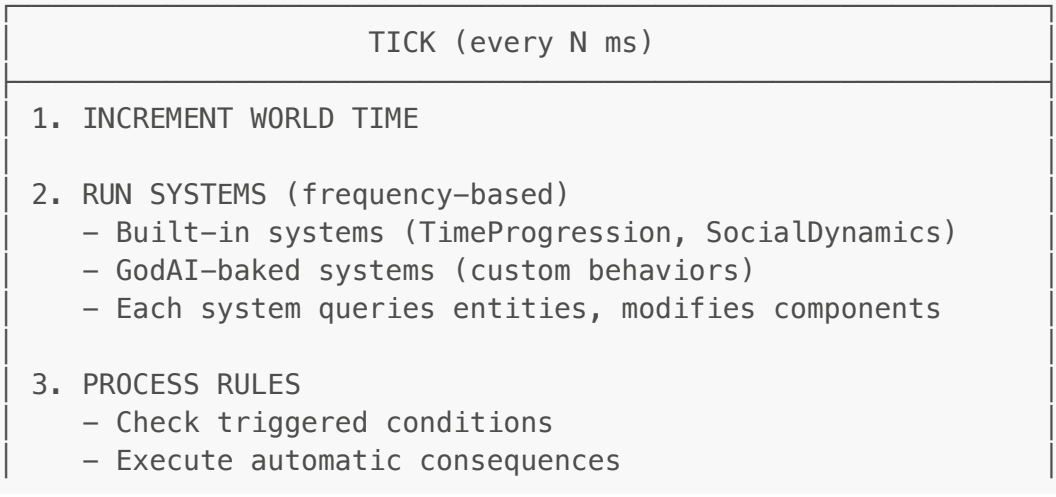
---

5. Simulation Flow

5.1 Building a Simulation from Prompt



5.2 Runtime Tick Loop



```
│    - Emit events                                                │
│                                                                 │
│ 4. AGENT COGNITION (every N ticks)                              │
│    For each active agent:                                       │
│    a. Gather perception (TextRenderer)                          │
│    b. Collect pending stimuli                                   │
│    c. LLM thinks based on perception + memory + goals           │
│    d. Agent chooses action                                      │
│    e. Execute action                                            │
│    f. Broadcast effects to nearby agents                        │
│                                                                 │
│ 5. CLEANUP                                                      │
│    - Remove expired entities                                    │
│    - Prune old perceptions/thoughts                             │
└─────────────────────────────────────────────────────────────────┘
```

---

6. Cognition Architecture

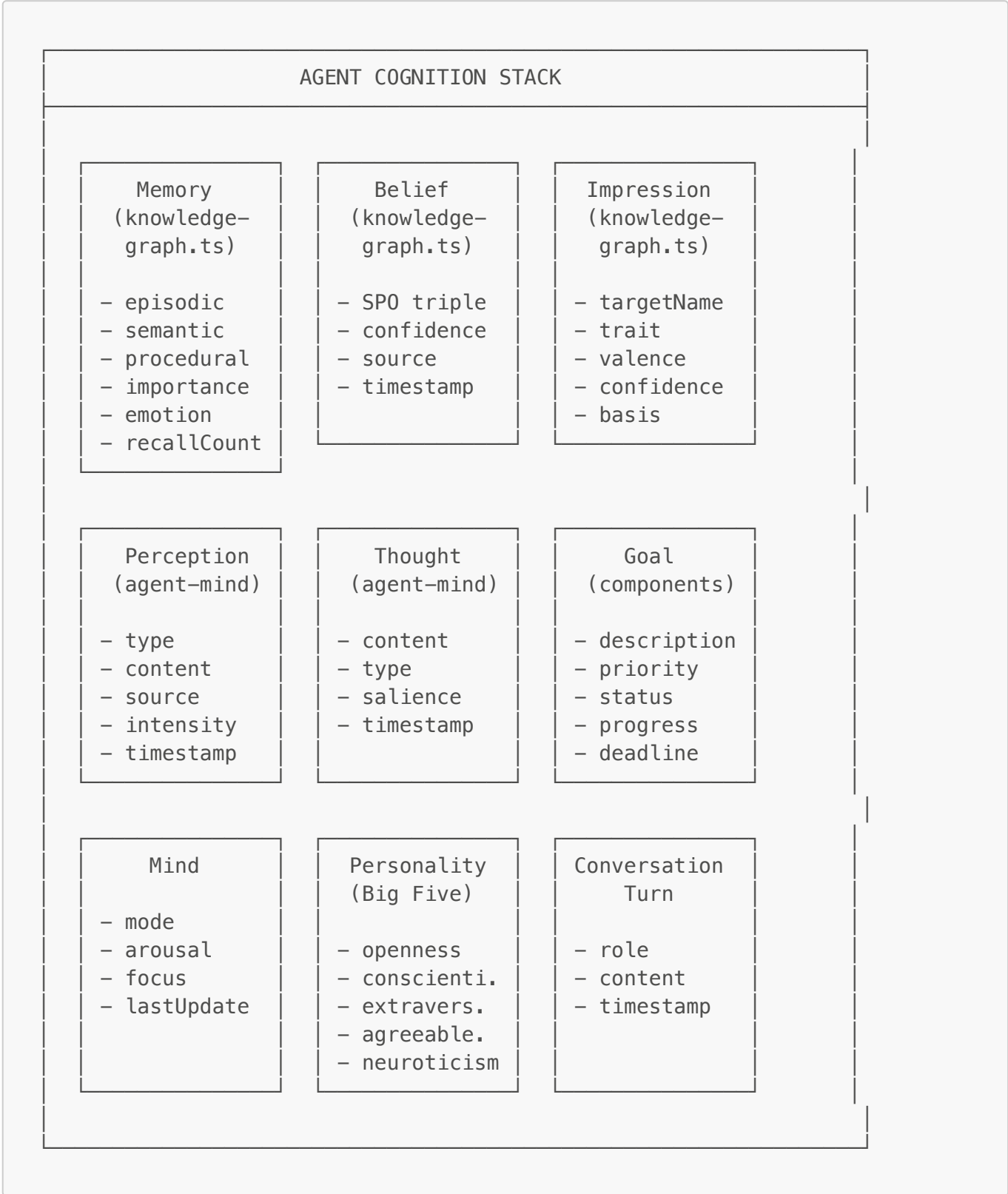
This section details how agent cognition fits into the world-building architecture and compares against the "Generative Agents" paper capabilities.

6.1 Generative Agents Comparison

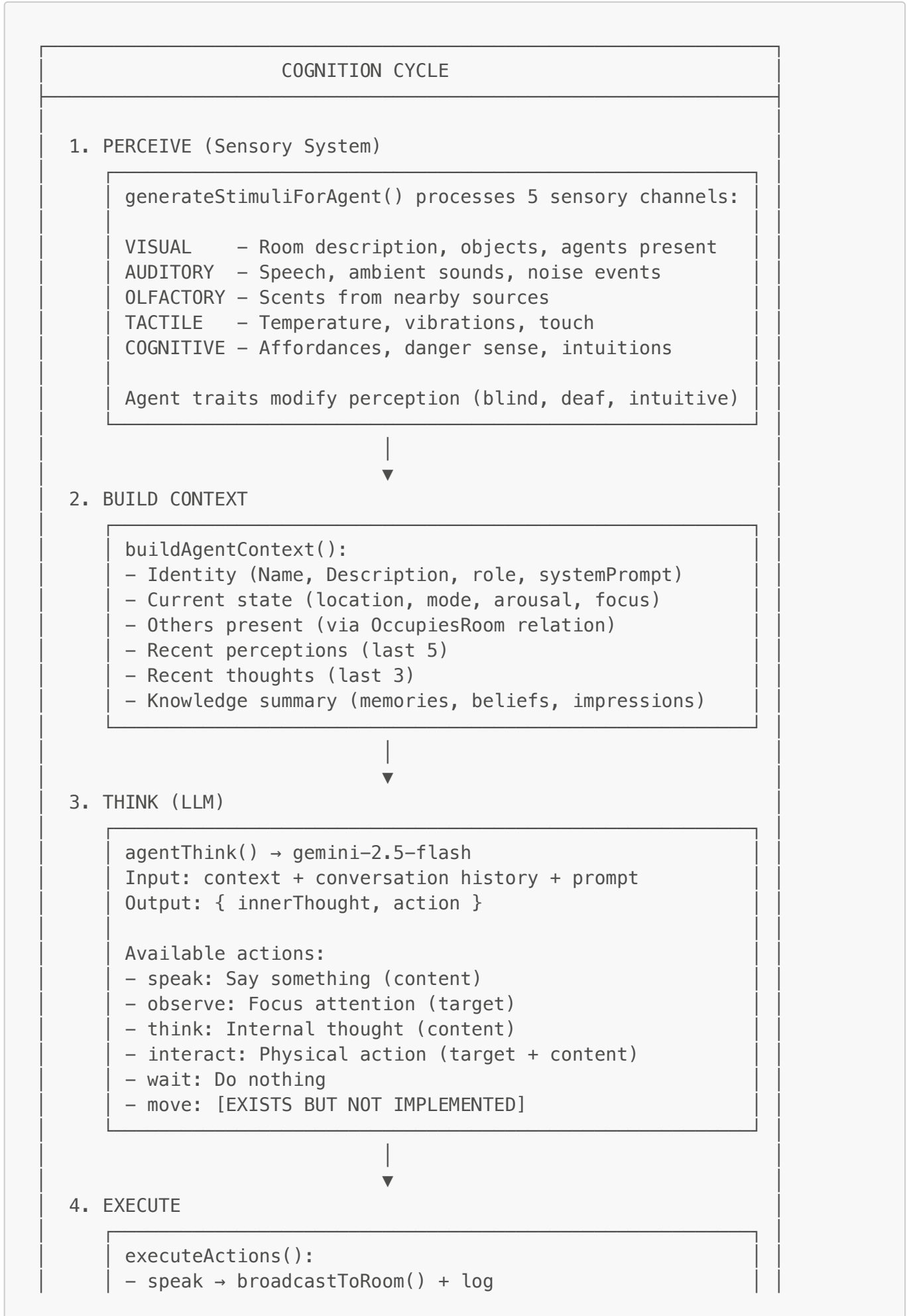
Capability	Generative Agents Paper	ArgOS Status	Notes
Memory	Episodic + semantic memories	✅ Implemented	`knowledge-graph.ts` - episodic/semantic/procedural types
Beliefs	Facts about the world	✅ Implemented	SPO triples with confidence scores
Impressions	Opinions of others	✅ Implemented	Trait-based with valence/confidence
Reflection	Periodic higher-order thoughts	⚠ Partial	Knowledge extraction exists, no scheduled reflection
Plans	Hierarchical plans with steps	❌ Missing	Goal component exists, no Plan component
Goal Pursuit	Goals → Plans → Actions	⚠ Partial	Goals exist but not used in thinking
Daily Schedule	Time-based routines	❌ Missing	No schedule component
Movement	Spatial navigation	⚠ Partial	Action type exists, not implemented
Environment Interaction	Object affordances	⚠ Partial	Object system exists, not wired to cognition

Capability	Generative Agents Paper	ArgOS Status	Notes
Identity/Personality	Consistent character	✅ Implemented	Agent.role, systemPrompt, Description, Personality
Conversation	Natural dialogue	✅ Implemented	ConversationTurn tracking, speech actions

6.2 Current Cognition Components



6.3 Cognition Flow



```
│     │ - observe → Mind.focus update                                  │ │
│     │ - think → log only                                             │ │
│     │ - interact → broadcastToRoom() + log                           │ │
│     └────────────────────────────────────────────────────────────────┘ │
│                                  │                                     │
│                                  ▼                                     │
│  5. LEARN                                                              │
│     ┌────────────────────────────────────────────────────────────────┐ │
│     │ extractKnowledgeFromInteraction():                             │ │
│     │ - LLM extracts memories, beliefs, impressions                  │ │
│     │ - Adds to agent's knowledge graph                              │ │
│     └────────────────────────────────────────────────────────────────┘ │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

6.4 Identified Gaps & Solutions

Gap 1: Goals Not Used in Thinking

Problem: `Goal` component exists with `HasGoal` relation, and `ctx.cognitive.createGoal()` API exists, but `agentThink()` doesn't read goals.

Solution: Update `buildAgentContext()` to include active goals:

```
// Add to buildAgentContext()
const goalTargets = getRelationTargets(world, eid, HasGoal);
const activeGoals = goalTargets
  .filter(gid => Goal.status[gid] === "active")
  .map(gid => ({
    description: Goal.description[gid],
    priority: Goal.priority[gid],
    progress: Goal.progress[gid]
  }));
```

Gap 2: No Plan Component

Problem: Generative Agents decomposes goals into plans with steps.

Solution: Add Plan component and system:

```
// New component
export const Plan = {
  goalEid: [] as number[],        // Links to Goal entity
  steps: [] as string[],          // JSON array of step descriptions
  currentStep: [] as number[],
  status: [] as string[],         // active, completed, failed
};

// GodAI can bake a PlanningSystem that:
```

```
// 1. Finds goals without plans
// 2. Generates steps via LLM
// 3. Tracks progress
```

Gap 3: Movement Not Implemented

Problem: `move` action type exists in `AgentAction` interface but `executeActions()` doesn't handle it.

Solution: Wire to pathfinding:

```
// In executeActions()
case "move":
  if (action.target) {
    const destRoom = findRoomByName(world, action.target);
    if (destRoom !== undefined) {
      removeComponent(world, eid, OccupiesRoom(rooms[0]));
      addComponent(world, eid, OccupiesRoom(destRoom));
      broadcastToRoom(world, rooms[0], {
        type: "departure",
        content: `${name} leaves toward ${action.target}`,
        source: name,
      }, eid);
    }
  }
  break;
```

Gap 4: Personality Not Used

Problem: `Personality` component (Big Five) exists but isn't included in `buildAgentContext()`.

Solution: Include personality in context:

```
const personality = hasComponent(world, eid, Personality) ? {
  openness: Personality.openness[eid],
  conscientiousness: Personality.conscientiousness[eid],
  extraversion: Personality.extraversion[eid],
  agreeableness: Personality.agreeableness[eid],
  neuroticism: Personality.neuroticism[eid],
} : null;

// Add to prompt: "PERSONALITY TRAITS: {traits}"
```

Gap 5: No Reflection Scheduling

Problem: Generative Agents triggered reflection when importance threshold exceeded.

Solution: Add reflection system:

```
// Bake this system via GodAI
function ReflectionSystem(world, ctx) {
  for (const eid of ctx.query(world, [Agent, Mind])) {
    const memories = ctx.cognitive.getMemories(world, eid);
    const totalImportance = memories.reduce((sum, m) => sum +
m.data.importance, 0);

    if (totalImportance > REFLECTION_THRESHOLD) {
      // Trigger reflection via LLM
      // Creates higher-level memory synthesizing recent experiences
    }
  }
}
```

Gap 6: TextRenderer Not Wired

Problem: `text-renderer.ts` exists but agents don't perceive via rendered text.

Solution: Replace raw stimuli with rendered perception:

```
// In cognition-system.ts, before cognition cycle
for (const eid of activeAgents) {
  const rooms = getRelationTargets(world, eid, OccupiesRoom);
  if (rooms.length > 0) {
    const perception = renderPerception(eid, rooms[0], {
      includeAffordances: true
    });
    addPerception(world, eid, {
      type: "room_observation",
      content: perception,
      source: "self",
    });
  }
}
```

6.5 GodAI Cognition Extensions

GodAI can extend cognition at runtime through these mechanisms:

1. New Cognitive Components

```
// GodAI can create new cognitive components
godCommand(state, `Create a "Motivation" component with:
  - drive: string (what motivates)
  - intensity: number (how strongly)
  - source: string (where it comes from)`);
```

2. New Cognitive Systems

```
// GodAI can bake new systems that process cognition
godCommand(state, `Create a system that:
  - Checks each agent's memories for traumatic events
  - If found, increases neuroticism temporarily
  - Triggers avoidance behavior near similar stimuli`);
```

3. Cognitive Context API

Systems have access to `ctx.cognitive` with:

- `createGoal(world, agentEid, data)` → create goal
- `createMemory(world, agentEid, data)` → create memory
- `createBelief(world, agentEid, data)` → create belief
- `createThought(world, agentEid, data)` → create thought
- `createImpression(world, agentEid, data)` → create impression
- `getGoals(world, agentEid)` → read all goals
- `getMemories(world, agentEid)` → read all memories
- `getBeliefs(world, agentEid)` → read all beliefs
- `updateGoal(eid, updates)` → modify goal
- `completeGoal(world, eid)` → mark goal complete

4. Persona System

Rich persona generation available via `persona.ts`:

- Long-term and short-term goals
- Motivations and fears
- Backstory and relationships
- Personality traits and quirks
- Speech patterns and catchphrases

6.6 Cognition Integration Roadmap

Phase 1: Complete Basic Cognition

- [ ] Wire goals into `buildAgentContext()`
- [ ] Wire personality into `buildAgentContext()`
- [ ] Implement `move` action
- [ ] Wire `TextRenderer` to perception

Phase 2: Enhanced Memory

- [ ] Add reflection scheduling system
- [ ] Add memory consolidation (short-term → long-term)
- [ ] Add forgetting curves

Phase 3: Planning

- [ ] Add `Plan` component
- [ ] Bake `PlanningSystem`
- [ ] Goal → Plan decomposition via LLM

Phase 4: Full Generative Agents Parity

- [ ] Daily schedule component
- [ ] Time awareness in cognition
- [ ] Location preference learning
- [ ] Relationship depth tracking

6.7 Agent Perception via TextRenderer (Target State)

Agents will perceive the world through TextRenderer, which produces MUD-style descriptions:

```
=== Village Square ===

The central square of a small medieval village. Cobblestones worn
smooth by countless feet. A well stands in the center.

Warm light flickers from nearby torches.

You see:
  - a wooden cart (open)
  - a market stall displaying wares
  - a stone well

People here:
  - Berta (a shrewd merchant)

You can:
  - examine <object>
  - take <object>
  - talk to Berta
  - go north (to Tavern)
  - go south (to Blacksmith)
```

This text becomes the agent's perception, which feeds into their cognitive loop.

6.8 Sensory System Implementation

The sensory system (`sensory-system.ts`) processes perception through five distinct channels:

```
┌─────────────────────────────────────────────────────────────┐
│                    SENSORY PROCESSING                       │
├─────────────────────────────────────────────────────────────┤
```

```
│                                                                          │
│  VISUAL Channel                                                          │
│  ├── Room description (TextRenderer)                                     │
│  ├── Objects present with states                                         │
│  ├── Other agents and their activities                                   │
│  └── Modified by: blind (blocks), keen_sight (enhances)                  │
│                                                                          │
│  AUDITORY Channel                                                        │
│  ├── Speech from other agents                                            │
│  ├── Ambient sounds (AmbientStimulusSystem)                              │
│  ├── Action sounds (doors, footsteps, etc.)                              │
│  └── Modified by: deaf (blocks), keen_hearing (enhances)                 │
│                                                                          │
│  OLFACTORY Channel                                                       │
│  ├── Scents from stimulus sources                                        │
│  ├── Environmental odors                                                 │
│  └── Modified by: anosmic (blocks)                                       │
│                                                                          │
│  TACTILE Channel                                                         │
│  ├── Temperature changes                                                 │
│  ├── Physical contact                                                    │
│  ├── Vibrations                                                          │
│  └── Less commonly modified                                              │
│                                                                          │
│  COGNITIVE Channel (Sixth Sense)                                         │
│  ├── Available affordances on nearby objects                             │
│  ├── Danger detection (hostile entities, traps)                          │
│  ├── Social intuitions                                                   │
│  └── Modified by: intuitive (enhances), psychic (full access)            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Stimulus Sources (Ambient Emitters)

Entities with `StimulusSource` component periodically emit stimuli:

```
// Example: A crackling fire
createStimulusSourceEntity(world, {
  name: "campfire",
  stimulusType: "sound",           // Maps to auditory
  template: "{name} crackles and pops",
  interval: 8000,                  // Every 8 seconds
  roomId: tavernRoom
});

// Example: A fragrant flower
createStimulusSourceEntity(world, {
  name: "rose bush",
  stimulusType: "scent",           // Maps to olfactory
  template: "The sweet scent of {name} fills the air",
  interval: 15000,
```

```
  roomId: gardenRoom
});
```

Cognitive Sense and Affordances

The cognitive channel provides agents with meta-knowledge about their world:

```
COGNITIVE PERCEPTION:
- You sense that the wooden door can be: opened, examined, knocked on
- You sense that the treasure chest can be: opened, examined, taken
- You notice Alice (nearby agent) can be: talked to, observed, followed
- You sense potential danger from the dark corridor
```

This allows agents to know what actions are available without explicitly examining each object.

---

7. Agent Hierarchy

7.1 Current: Two-Tier GodAI + NPCs

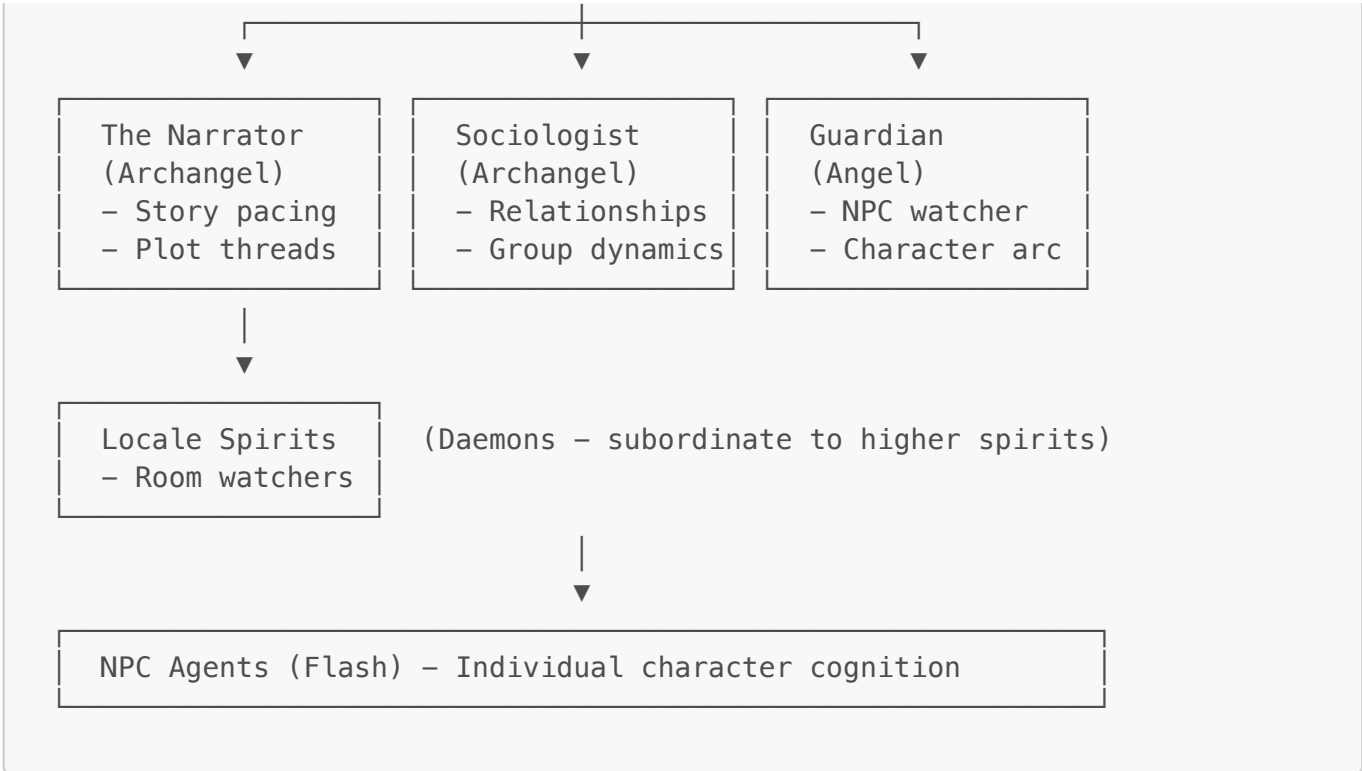
```
┌─────────────────────────────────────────────────────────┐
│  GodAI                                                  │
│  ├── Planner (Pro/Opus) - designs simulations           │
│  └── Executor (Flash) - builds via tools                │
└─────────────────────────────────────────────────────────┘
                          │
                          │ creates
                          ▼
┌─────────────────────────────────────────────────────────┐
│  NPC Agents (via createAgent)                           │
│  - Each has Agent + Mind components                     │
│  - Cognition system processes all agents each tick      │
│  - LLM-based thinking (gemini-2.5-flash)                │
│  - Actions: speak, observe, think, interact, wait       │
└─────────────────────────────────────────────────────────┘
```

7.2 Specialized Agent Hierarchy (Implemented)

Status: ✅ **IMPLEMENTED** - See **Section 13: Spirit Hierarchy** for full details.

The Spirit Hierarchy implements an emanationist architecture where GodAI delegates observation and steering to specialized AI spirits:

```
┌─────────────────────────────────────────────────────────┐
│  GodAI (Opus) - High-level design, narrative direction  │
└─────────────────────────────────────────────────────────┘
                          │
                          │ commands & receives reports
```



Key Features:

- Spirits observe ECS state and report to superiors
- The Narrator (first implemented) tracks narrative structure, tension, and pacing
- Spirits can intervene by injecting stimuli, sending intuitions, or modifying moods
- DivineMessage protocol enables inter-spirit communication
- Full test coverage (43 unit + integration tests)

---

8. Component & System Catalogs

8.1 Built-in Components

Component	Purpose	Key Fields
`Name`	Entity identifier	`value: string`
`Description`	Text description	`text: string`
`Agent`	Marks cognitive entity	`active, role, systemPrompt`
`Mind`	Agent mental state	`arousal, focus, mode`
`Thought`	Agent's current thought	`content, type, salience`
`Perception`	Sensory input	`type, content, source`
`Memory`	Stored memory	`content, importance`
`Goal`	Agent objective	`description, priority`
`Room`	Location marker	`capacity, ambience`
`Position`	2D coordinates	`x, y`

8.2 WorldSchema Components

Component	Purpose
`ObjectType`	Links entity to schema prefab
`ObjectState`	Current state in state machine
`Traits`	Capability flags (openable, takeable, etc.)
`LightSource`	Emits light with intensity
`Container`	Can hold other entities

8.3 Built-in Systems

System	Purpose	Frequency
`TimeProgression`	Advances world time	10s
`SocialDynamics`	Agent arousal from presence	10s
`AmbientStimulusSystem`	Emits periodic stimuli from StimulusSource entities	5s
`MindDecay`	Decays arousal over time	5s
`AgentCognition`	Processes agent thinking via sensory system	10s

8.4 Dynamic (AI-Created)

GodAI can create any component or system at runtime:

```
// Component
createComponent("Temperature", { current: "number", max: "number" })

// System
bakeNewSystem("HeatTransfer", {
  description: "Heat flows between adjacent entities",
  frequency: 5000,
  logic: "..."
})
```

---

9. Feasibility & Constraints

9.1 What Works Well

- **Medium-complexity simulations** - Economy, predator-prey, social dynamics
- **Component/entity creation** - GodAI reliably creates data structures
- **System baking** - Generated code usually works (with auto-fix loops)
- **Design-then-execute pattern** - Pro thinks, Flash builds

9.2 Challenges

- **Very novel concepts** - Struggles without examples
- **Complex emergent systems** - May need iteration/guidance
- **Long-running coherence** - Context limits require summarization
- **Debugging baked systems** - Generated code can be opaque

9.3 Guardrails

- Baked systems validated before execution
- Runtime errors caught and logged
- Auto-fix loop attempts repairs
- Invalid component access returns undefined (no crash)

---

10. Integration Roadmap

Phase 1: Connect WorldSchema to GodAI (Priority)

- ☐ Add `spawn` tool - instantiate from prefab
- ☐ Add `defineObjectType` tool - extend schema
- ☐ Add `defineAffordance` tool - add actions
- ☐ Add `defineRule` tool - add reactive rules

Phase 2: Wire TextRenderer to Cognition ✅ COMPLETE

- ☑ Agent perception uses TextRenderer.renderPerception()
- ☑ Affordances shown as available actions (via cognitive sense)
- ☑ Agents see MUD-style text, not raw components
- ☑ Sensory system with 5 modalities (visual, auditory, olfactory, tactile, cognitive)
- ☑ Effect executor for affordance-based state changes
- ☑ Ambient stimulus sources for periodic environmental stimuli

Phase 3: Unified Simulation Runner

- ☐ Single entry point: prompt → running simulation
- ☐ Built-in UI for observation
- ☐ REPL for GodAI commands during runtime

Phase 4: Enhanced Narrative

- ☐ Plot arc tracking
- ☐ Character relationship visualization
- ☐ Narrative tension/pacing systems

Phase 5: GodAI Monitoring & Steering ✅ IMPLEMENTED

- ☑ **GodAI Observation Loop** - Periodic world state inspection
  - `getWorldSummary()` - Monitor agent behaviors, emotional states, relationships
  - `getNarrativeTension()` - Track narrative arc progression

  - `getSteeringRecommendations()` - Detect stagnation, conflicts, emergent patterns
- ☑ **Event Injection** - GodAI can trigger events at will
  - `injectEnvironmentalEvent()` - Environmental events to rooms
  - `injectIntuition()` - Cognitive sense events to agents
  - `broadcastAnnouncement()` - World-wide announcements
- ☑ **Narrative Steering** - Guide without micromanaging
  - `setNarrativeGoals()` / `getNarrativeGoals()` - Track narrative direction
  - `modifyAgentMood()` - Influence agent behavior
  - `pauseAgent()` / `resumeAgent()` - Control agent activity
- ☑ **Dynamic System Creation** - Bake new systems mid-simulation (via existing `bakeNewSystem` tool)
- ☑ **Simulation Analytics** - Real-time metrics
  - `getInterventionStats()` - Track intervention count, tension, stagnation
  - Steering patterns: nudge, catalyst, escalation, resolution
  - World state summaries with text output for GodAI context

---

11. File Map

```
src/
├── god/
│   ├── god-agent.ts       # Main GodAI implementation
│   ├── system-baker.ts    # Code generation for systems
│   └── monitoring-system.ts # World observation & narrative steering
├── ecs/
│   ├── world.ts           # World creation
│   ├── components.ts      # Built-in components
│   ├── relations.ts       # Entity relationships
│   ├── dynamic-components.ts  # Runtime component creation
│   ├── dynamic-systems.ts # Runtime system management
│   └── tools.ts           # GodAI tool implementations
├── world/
│   ├── schema.ts          # WorldSchema definitions
│   ├── object-manager.ts  # Prefab instantiation
│   ├── rules-engine.ts    # Declarative rules
│   ├── effect-executor.ts # Affordance effect execution
│   └── text-renderer.ts   # MUD-style output
├── cognition/
│   ├── agent-mind.ts      # Agent LLM processing
│   ├── cognition-system.ts # Agent tick processing
│   ├── sensory-system.ts  # Sensory perception processing
│   └── knowledge-graph.ts # Memory/knowledge
├── spirits/
│   ├── index.ts           # Module exports
│   ├── types.ts           # Spirit types and interfaces
│   ├── spirit-registry.ts # Hierarchy management & messaging
│   ├── spirit-cognition.ts # Spirit LLM processing loop
│   ├── spirit-system.ts   # ECS ticker for spirits
│   ├── narrator-spirit.ts # The Narrator archangel definition
│   ├── consistency-spirit.ts # The Arbiter - validates & routes reports
```

```
✅ NEW
│   ├── agent-daemon.ts    # Personal agent daemons (protection +
challenge) ✅ NEW
│   └── story-templates.ts # Story arc templates (3-act, mystery, etc.)
├── introspection/
│   └── introspection.ts   # Dynamic registries & event buffer ✅ NEW
├── llm/
│   └── config.ts          # Centralized LLM model config (LOCKED) ✅ NEW
├── systems/
│   └── ambient-stimulus-system.ts # Periodic stimulus emission
└── behavioral-tests/
    ├── challenge-01-economy.ts     # Economy simulation test
    ├── challenge-02-predator-prey.ts # Ecosystem test
    ├── 03-introspection-stress.ts  # Introspection system stress test ✅
NEW
    ├── 04-spirit-godai-integration.ts # Spirit → GodAI integration test
✅ NEW
    └── 05-daemon-routing-integration.ts # Daemon & routing test ✅ NEW
```

---

12. GodAI Monitoring & Steering (Design)

This section details the design for GodAI to actively monitor and guide simulations.

12.1 Observation Loop Architecture

```
┌─────────────────────────────────────────────────────────────────────┐
│                    GODAI OBSERVATION LOOP                           │
│                    (runs every N simulation ticks)                  │
├─────────────────────────────────────────────────────────────────────┤
│                                                                     │
│  1. COLLECT WORLD STATE                                             │
│     ├── Query all agents: location, mood, recent actions            │
│     ├── Query relationships: who knows whom, attitudes              │
│     ├── Query environment: room states, object states               │
│     └── Summarize into GodAI-consumable context                     │
│                                                                     │
│  2. ANALYZE                                                         │
│     ├── Detect stagnation (no actions, repetitive behavior)         │
│     ├── Identify emergent patterns (factions, romances, feuds)      │
│     ├── Track narrative tension (conflict level, stakes)            │
│     └── Compare against narrative goals                             │
│                                                                     │
│  3. DECIDE                                                          │
│     ├── Should I intervene? (based on narrative goals)              │
│     ├── What type of intervention?                                  │
│     │   ├── Subtle: environmental change, NPC mood shift            │
│     │   ├── Moderate: new NPC arrival, quest trigger                │
│     │   └── Major: crisis event, system-level change                │
│     └── How to preserve emergent narrative?                         │
│                                                                     │
```

```
│  4. INTERVENE (via existing tools)                                    │
│     ├── injectEvent() - Direct stimulus to room/agent                 │
│     ├── createAgent() - New NPC enters the scene                      │
│     ├── modifyComponent() - Change agent states                       │
│     ├── bakeNewSystem() - Add new behaviors to world                  │
│     └── broadcast() - Narrate to all agents                           │
│                                                                       │
└──────────────────────────────────────────────────────────────────────┘
```

12.2 New Tools for GodAI Monitoring

Tool	Purpose	Implementation
`getWorldSummary()`	Compressed world state for GodAI context	Query all entities, summarize
`getAgentStatus(eid)`	Detailed agent state (mood, focus, goals)	Read agent components
`getRelationshipGraph()`	Who knows whom, with attitudes	Query Impression/Belief stores
`getNarrativeTension()`	Calculated conflict/stakes metric	Analyze recent events
`injectEvent(roomId, event)`	Trigger stimulus in a room	broadcastToRoom()
`setNarrativeGoal(goal)`	Tell GodAI what story to aim for	Store in GodAgent component
`pauseAgent(eid)` / `resumeAgent(eid)`	Temporarily disable agent	Set Agent.active

12.3 GodAgent Component Extension

```
// Extended GodAgent component
GodAgent: {
  worldName: string,
  narrative: string,           // Current narrative summary
  narrativeGoals: string[],    // What GodAI is trying to achieve
  tension: number,             // Current narrative tension (0-1)
  lastObservation: number,     // Timestamp of last observation
  interventionCount: number,   // Track how often GodAI intervenes
  observationInterval: number, // How often to observe (ms)
}
```

12.4 Event Injection System

GodAI can inject events at various levels:

```
// Room-level event (all agents in room perceive)
injectEvent({
  type: "environmental",
  room: tavernRoom,
  content: "A mysterious stranger enters the tavern, cloaked in shadow",
  modality: "visual",
});

// Agent-specific event (only one agent perceives)
injectEvent({
  type: "intuition",
  target: heroAgent,
  content: "You have a strong feeling that danger approaches",
  modality: "cognitive",
});

// World-level event (broadcast to all agents)
injectEvent({
  type: "announcement",
  broadcast: true,
  content: "The church bells toll midnight across the village",
  modality: "auditory",
});
```

12.5 Narrative Steering Patterns

Pattern 1: Gentle Nudges

```
Observation: Two agents haven't interacted despite proximity
Action: Create ambient stimulus that brings them together
Example: "The sudden rain forces everyone under the tavern awning"
```

Pattern 2: Catalyst Introduction

```
Observation: Story has stagnated, no conflict
Action: Introduce new NPC with conflicting goals
Example: Create agent with opposing faction membership
```

Pattern 3: Crisis Escalation

```
Observation: Conflict exists but tension is low
Action: Raise stakes via environmental pressure
Example: "Food supplies are running low" (add Scarcity system)
```

Pattern 4: Resolution Facilitation

```
Observation: Conflict has gone on too long
Action: Provide opportunity for resolution
Example: Create neutral ground/mediator NPC
```

12.6 World State Summarization

For GodAI to make decisions, it needs compressed world state:

```
interface WorldStateSummary {
  tick: number;
  agentSummaries: {
    name: string;
    location: string;
    mood: string;        // Derived from Mind.arousal + recent actions
    recentActions: string[];
    relationships: { name: string; attitude: string }[];
  }[];
  activeConflicts: {
    parties: string[];
    nature: string;
    intensity: number;
  }[];
  environmentState: {
    roomName: string;
    description: string;
    ambience: string;
    objectStates: { name: string; state: string }[];
  }[];
  narrativeArc: {
    currentPhase: string;  // "setup" | "rising" | "climax" | "falling" |
"resolution"
    tension: number;
    recentEvents: string[];
  };
}
```

12.7 Implementation Priority

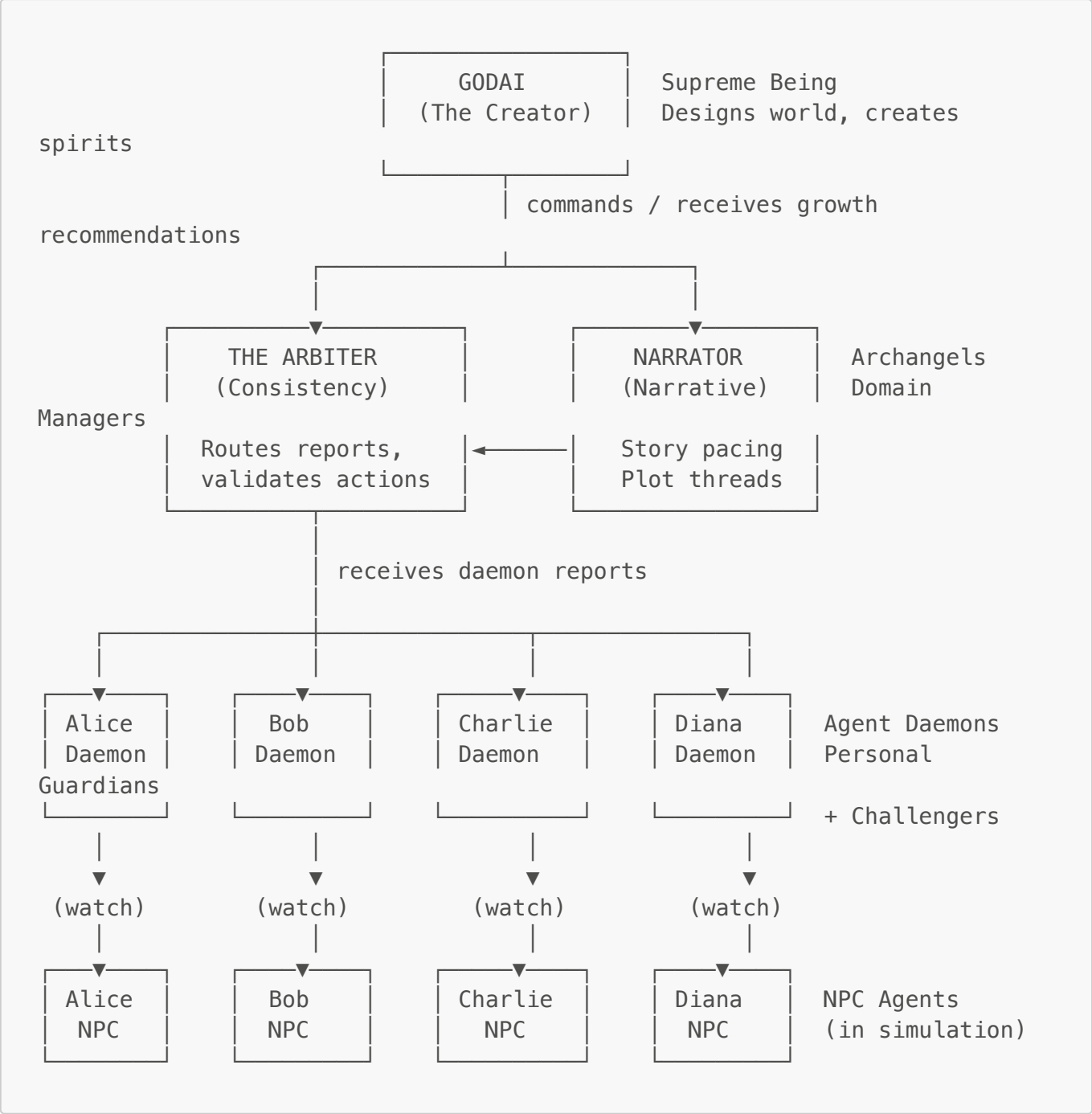
1. `getWorldSummary()` - Foundation for all monitoring ✅
2. `injectEvent()` - Basic intervention capability ✅
3. **Observation loop scheduler** - Periodic GodAI cognition ✅
4. **Narrative tension calculation** - Inform intervention decisions ✅
5. **Steering patterns** - Higher-level intervention logic ✅

---

13. Spirit Hierarchy (Implemented)

The Spirit System provides a celestial hierarchy of AI agents that observe and steer the simulation, inspired by emanationist/Kabbalistic cosmology. Spirits report up to GodAI while managing their domains.

13.1 Hierarchy Structure (Updated)



Key Features of the Updated Hierarchy:

1. **GodAI** receives growth recommendations (not just concerns) for world expansion
2. **The Arbiter** (ConsistencySpirit) is the central hub for routing all daemon reports
3. **Agent Daemons** serve dual purpose: protection AND growth challenges
4. **The Narrator** receives narrative-domain issues from The Arbiter
5. Future spirits (Sociologist, Ecologist, etc.) will receive their domain-specific issues

13.2 Spirit Types

Rank	Role	Capabilities
Archangel	Domain manager	Inject events, modify mood, send reports, route reports
Angel	Local/entity manager	Inject events to specific area, reports
Daemon	Task-specific	Observe, whisper to agents, report to superior
Agent Daemon	Personal guardian	Watch individual agent, whisper guidance AND challenge

13.2.1 Agent Daemons (Personal Guardian Spirits) ✅ NEW

Each agent in the simulation has a personal daemon that serves a **dual purpose**:

Protection Mode (Guidance Whispers):

- Detects concerns: stuck agents, low arousal, high arousal, danger, goal drift
- Sends gentle guidance via cognitive stimuli ("inner voice")
- Reports urgent concerns to higher spirits (The Arbiter)

Growth Mode (Challenge Whispers):

- Detects growth opportunities: too comfortable, ready for challenge, stagnating, needs conflict
- Sends provocative challenges via cognitive stimuli to push growth
- Reports growth opportunities to GodAI for world-level challenge creation

```
// Daemon observes and detects BOTH concerns AND growth opportunities
interface DaemonObservation {
  agentName: string;
  currentState: AgentStateSnapshot;
  concerns: DaemonConcern[];           // Protection mode
  growthOpportunities: GrowthOpportunity[];  // Challenge mode
  achievements: string[];
}

// Growth opportunity types
type GrowthOpportunity = {
  type: "too_comfortable" | "ready_for_challenge" | "stagnating" |
        "needs_conflict" | "breakthrough_possible" | "skill_plateau" |
        "relationship_test";
  description: string;
  suggestedChallenge: string;
  urgency: "low" | "medium" | "high";
}
```

Challenge Whisper Types:

Type	Purpose	Example
`provocation`	Plant doubt, stir jealousy	"Are you truly content with this?"
`doubt`	Question abilities	"Is this really the best you can do?"

Type	Purpose	Example
`ambition`	Whisper of greater things	"You were meant for greater things"
`curiosity`	Hint at mysteries	"What lies beyond?"
`restlessness`	Create need to change	"Something is wrong. You need to move."

13.2.2 The Arbiter (ConsistencySpirit) ✅ NEW

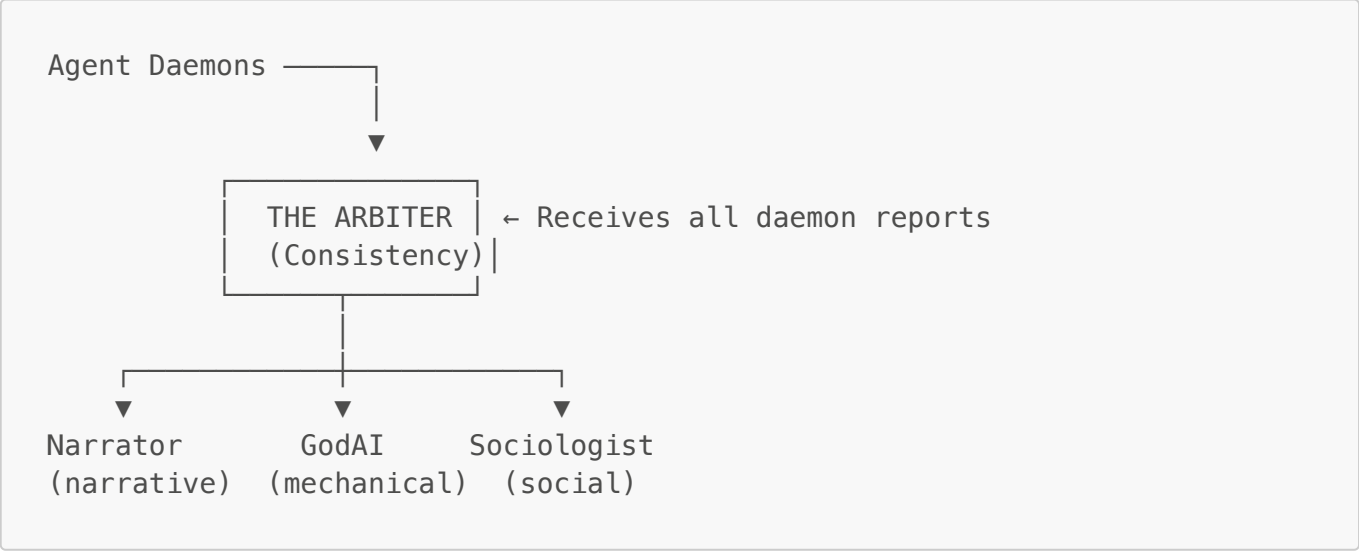
The Arbiter is a special archangel responsible for:

1. **Dynamic Introspection**: Maintains awareness of ALL actions, components, systems via dynamic registries
2. **Validation**: Validates agent actions and narrative events against world state
3. **Report Routing**: Routes daemon reports and issues to appropriate spirits:
   - **Narrative issues** → The Narrator (narrative domain)
   - **Mechanical issues** → GodAI (for system fixes/additions)
   - **Social issues** → Sociologist (when implemented)

Issue Categories:

Category	Examples	Routed To
`narrative`	"Dragon mentioned but not established", plot holes	Narrator
`mechanical`	Impossible actions, missing systems, broken rules	GodAI
`social`	Relationship inconsistencies, faction issues	Sociologist
`environmental`	Weather anomalies, resource inconsistencies	Ecologist

Report Routing Flow:



13.3 Spirit Domains

- **guardian**: The Arbiter - validates consistency, routes reports ✅ NEW
- **narrative**: Plot threads, dramatic tension, pacing, story beats

- **social**: Relationships, factions, conflicts, social dynamics
- **ecology**: Environment, weather, resources, space
- **economy**: Trade, markets, resource flows
- **watcher**: Watches over specific NPCs (via Agent Daemons) ✅ NEW
- **locale**: Manages specific locations

13.4 Message Protocol (DivineMessage)

Spirits communicate via structured messages:

```
interface DivineMessage {
  id: string;
  timestamp: number;
  from: number;        // Spirit entity ID
  to: number;          // Target spirit entity ID
  type: MessageType;   // "report" | "directive" | "alert" | "broadcast"
  domain: SpiritDomain;
  priority: MessagePriority;  // "low" | "normal" | "high" | "urgent"
  subject: string;
  content: string;
  requiresResponse: boolean;
}
```

13.5 The Narrator Spirit

The first implemented spirit, **The Narrator**, is an archangel of the narrative domain:

Responsibilities:

- Track story structure (three-act, tension curves)
- Identify stagnation and pacing issues
- Track plot threads and character arcs
- Identify protagonists and antagonists
- Suggest and execute interventions

Narrative State Tracking:

```
interface NarrativeState {
  currentAct: number;        // 1, 2, 3
  currentPhase: NarrativePhase;  // "exposition" | "inciting" | "rising" |
...
  tension: number;           // 0-1
  plotThreads: PlotThread[];
  protagonists: string[];
  antagonists: string[];
  characterArcs: CharacterArc[];
}
```

13.6 Spirit Cognition Loop

Each spirit follows this cycle:

```
1. OBSERVE ECS
   ├── Collect agent snapshots (location, mood, actions)
   ├── Collect room snapshots (occupants, ambience)
   └── Review recent actions and events

2. ANALYZE (LLM)
   ├── Interpret observations through domain lens
   ├── Identify significant patterns
   └── Decide on actions

3. REPORT
   ├── Send observations to superior
   └── Flag urgent issues

4. INTERVENE (if authorized)
   ├── Inject environmental stimuli
   ├── Send intuitions to agents
   └── Modify mood states
```

13.7 GodAI Spirit Tools

GodAI has these tools for managing spirits:

Tool	Description
`getSpiritHierarchy`	View the current spirit hierarchy
`getSpiritReports`	Get pending reports from spirits
`sendDirectiveToSpirit`	Command a spirit to take action
`createSpirit`	Create a new spirit for a domain
`getSpiritObservations`	Get a spirit's recent observations
`getNarratorState`	Get the Narrator's narrative analysis
`tickSpirits`	Force an immediate spirit cognition cycle

13.8 Integration with Simulation

```
// Initialize spirit system with simulation
const spiritState = initializeSpiritSystem(world, {
  godAgentEid: godAgent.eid,
  autoCreateNarrator: true,  // Creates The Narrator automatically
});
```

```
// Start spirit observation cycles
startSpiritSystem();

// In main loop, tick spirits alongside simulation
await tickSpiritSystem(world, entityRegistry);
```

13.9 Story Arc Templates

The Narrator can follow pre-defined story templates that guide narrative structure. Templates define:

Available Templates:

Template	Genre	Description
`classic_three_act`	adventure	Setup → Confrontation → Resolution
`mystery`	mystery	Crime → Investigation → Solution
`slice_of_life`	slice_of_life	Low-tension character-focused narrative
`conflict`	conflict	Opposition → Escalation → Confrontation

Template Components:

- **Story Beats**: Key moments (inciting incident, midpoint, climax, etc.)
- **Tension Curves**: Target tension levels for each narrative phase
- **Character Roles**: Required and optional character types (protagonist, antagonist, mentor)
- **Pacing Guidelines**: When and how to intervene if story stagnates
- **Intervention Suggestions**: Context-specific nudges to keep the story moving

GodAI Story Template Tools:

Tool	Description
`listStoryTemplates`	View available narrative templates
`setStoryTemplate`	Activate a template for the simulation
`getStoryTemplateStatus`	Check alignment with template and get recommendations
`markStoryBeat`	Mark a story beat as completed
`getTemplateInterventions`	Get suggested interventions based on template
`suggestCharacterRoles`	Get role assignment suggestions for agents

Usage:

```
// GodAI activates a template
setStoryTemplate({ templateId: "mystery" });

// Check progress and get recommendations
const status = getStoryTemplateStatus();
```

```
// Returns: { alignment: 0.85, nextBeat: "first_clue", recommendations:
[...] }

// Mark story beats as they occur
markStoryBeat({ beatId: "discovery" });
markStoryBeat({ beatId: "detective_involved" });

// Get intervention suggestions when story stagnates
const interventions = getTemplateInterventions({ includeAllSources: true
});
```

---

14. Dynamic Spirit Creation (Design) 🔮 PROPOSED

This section explores how GodAI could dynamically create new spirits to manage emerging complexity.

14.1 Vision: Self-Expanding Hierarchy

As the simulation grows in complexity, GodAI should be able to:

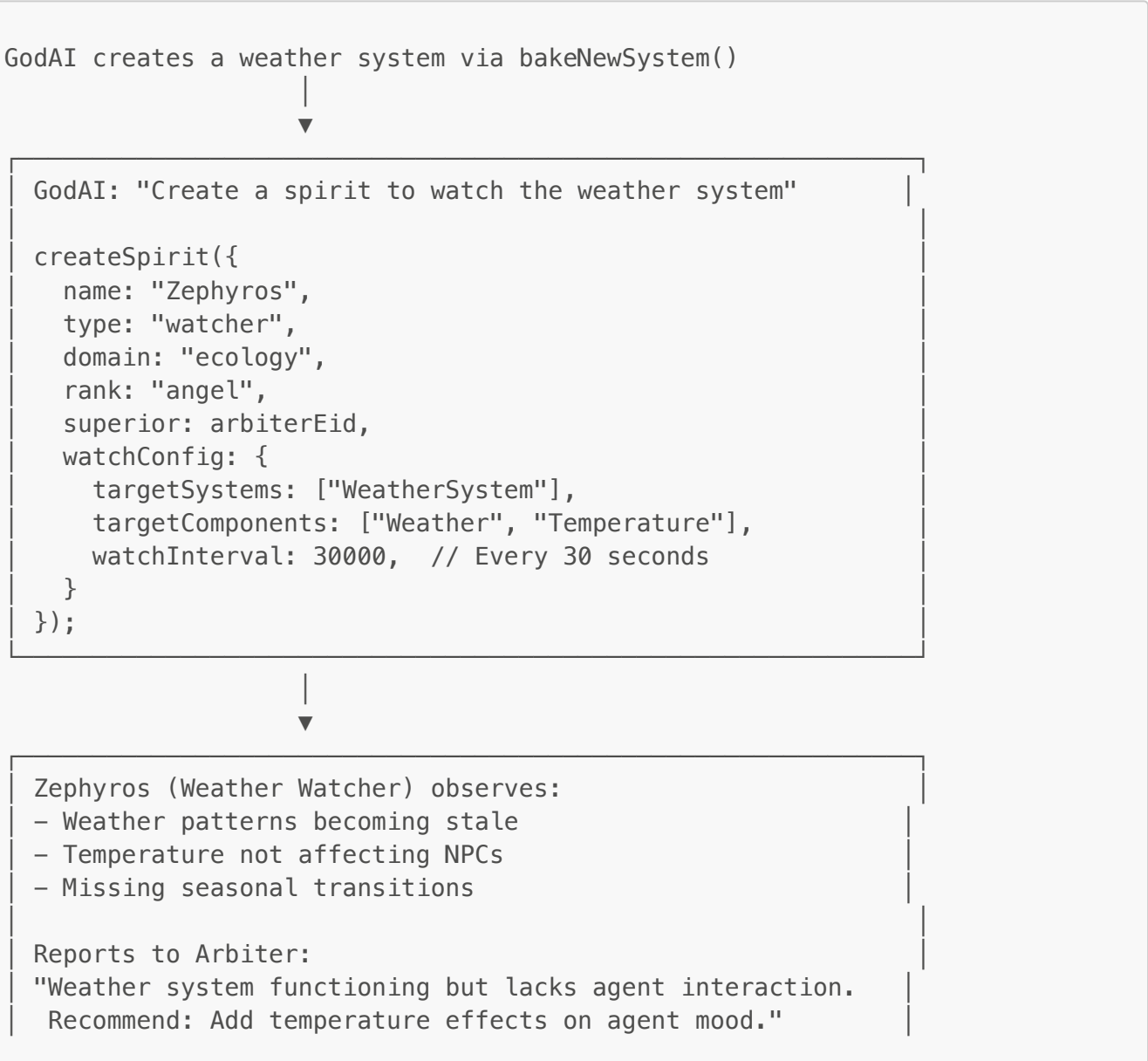
1. **Create System Watchers**: Spirits that observe specific baked systems
    - WeatherSpirit watching a weather system
    - PhysicsSpirit monitoring physics simulation
    - EconomySpirit tracking market dynamics
2. **Create Higher Angels**: Spirits that can themselves architect new subsystems
    - CraftingArchitect that designs and proposes crafting systems
    - QuestDesigner that creates quest structures and objectives
    - DungeonMaster that builds challenge areas
3. **Create Domain Coordinators**: Meta-spirits that manage groups of spirits
    - EnvironmentCoordinator managing Weather + Physics + Ecology spirits
    - SocialCoordinator managing Faction + Relationship + Politics spirits

14.2 Spirit Factory Design

```
// GodAI tool for dynamic spirit creation
interface CreateSpiritParams {
  name: string;
  type: "watcher" | "manager" | "architect" | "coordinator";
  domain: SpiritDomain;
  rank: "archangel" | "angel" | "daemon";
  superior?: number;  // Entity ID of superior spirit

  // For watchers - what to observe
  watchConfig?: {
    targetSystems?: string[];      // System names to watch
    targetEntities?: number[];     // Specific entities
```

```
    targetComponents?: string[];   // Component types
    watchInterval?: number;        // How often to observe
  };

  // For architects - what they can create
  architectConfig?: {
    canProposeSystems?: boolean;   // Can suggest new systems
    canProposeEntities?: boolean;  // Can suggest new entities
    canProposeRules?: boolean;     // Can suggest new rules
    proposalApproval?: "auto" | "godai" | "user";  // Who approves
  };

  // For coordinators - who they manage
  coordinatorConfig?: {
    subordinates?: number[];       // Spirit entity IDs
    canCreateSubordinates?: boolean;
  };
}
```

14.3 Example: Weather System Watcher



```
└──────────────────────────────────────────────────────────────┘
                          │
                          ▼
┌──────────────────────────────────────────────────────────────┐
│ Arbiter routes to GodAI:                                     │
│ "Mechanical enhancement needed: Weather → Agent effects"     │
│                                                              │
│ GodAI decides to bake TemperatureEffectSystem                │
└──────────────────────────────────────────────────────────────┘
```

14.4 Example: Quest Architect Angel

```
// GodAI creates a quest architect spirit
createSpirit({
  name: "The Questmaster",
  type: "architect",
  domain: "narrative",
  rank: "angel",
  superior: narratorEid,  // Reports to The Narrator
  architectConfig: {
    canProposeSystems: true,
    canProposeEntities: true,
    canProposeRules: true,
    proposalApproval: "godai",  // GodAI must approve proposals
  }
});
```

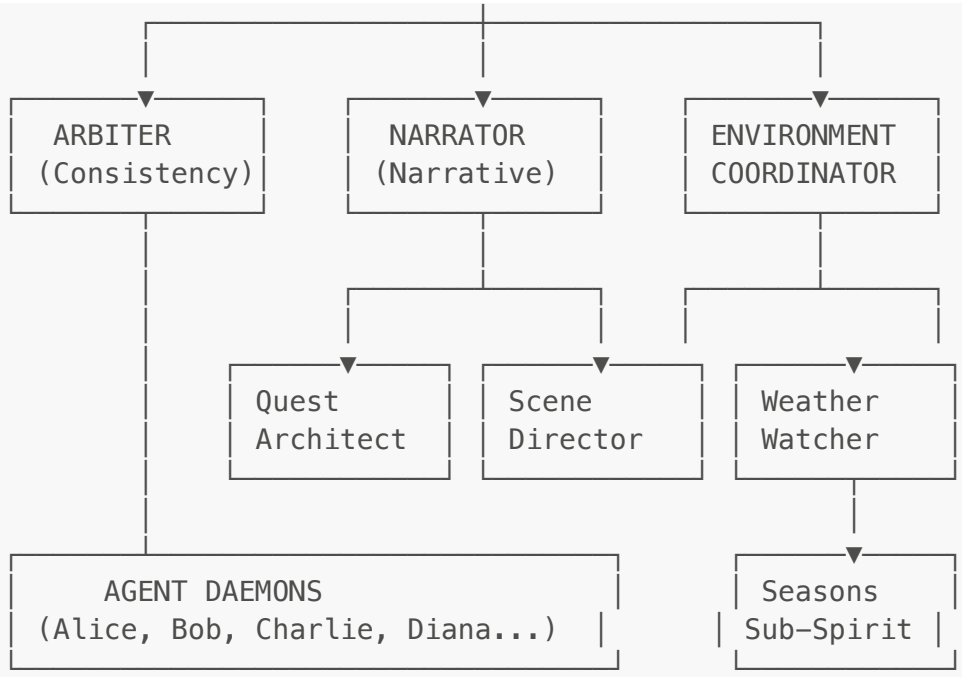
Questmaster's Cognition Loop:

```
1. OBSERVE: Current narrative state, agent goals, world state
2. ANALYZE: What quests would enhance the story?
3. PROPOSE: Submit quest design to GodAI
   - "Propose: Create 'The Missing Merchant' quest"
   - "Requires: QuestTracker component, RewardSystem"
   - "Entities: Quest giver NPC, clue objects, reward chest"
4. AWAIT: GodAI approval
5. EXECUTE: If approved, create quest infrastructure
```

14.5 Hierarchical Self-Organization

The ultimate vision is a self-organizing hierarchy:

```
                    ┌──────────────┐
                    │    GODAI     │
                    │ (Supreme)    │
                    └──────┬───────┘
                           │ creates & receives proposals
```



14.6 Benefits

1. **Scalability**: Complexity is managed by delegation, not centralization
2. **Domain Expertise**: Spirits specialize in their domains
3. **Emergent Organization**: Hierarchy grows organically based on needs
4. **Reduced GodAI Load**: Lower spirits handle routine observation/steering
5. **Richer World**: More eyes watching = more opportunities discovered

14.7 Implementation Roadmap

Phase	Feature	Priority
1	`createSystemWatcher` tool	High
2	System Watcher cognition loop	High
3	`createArchitectSpirit` tool	Medium
4	Proposal/approval workflow	Medium
5	`createCoordinatorSpirit` tool	Low
6	Self-organization rules	Low

---

15. Dynamic Introspection System ✅ NEW

The introspection system provides real-time awareness of all actions, components, and systems.

15.1 Registries

```
// Action Registry - ALL actions available in the simulation
const ACTION_REGISTRY: Record<string, ActionMetadata> = {
```

```
  "speak": { domain: "social", canFail: false },
  "move": { domain: "navigation", requiresTarget: true },
  "attack": { domain: "combat", canFail: true, dangerous: true },
  "craft": { domain: "production", requiresTarget: true, producesItem:
true },
  // ... 15 actions total, dynamically populated
};

// Component Registry - ALL components in the simulation
const COMPONENT_REGISTRY: ComponentMetadata[] = [
  { name: "Agent", category: "identity" },
  { name: "Mind", category: "cognition" },
  { name: "Health", category: "stats" },
  { name: "Inventory", category: "equipment" },
  // ... 30 components total, dynamically populated
];
```

15.2 Rolling Event Buffer

Tracks recent events for pattern detection:

```
interface RollingEventBuffer {
  events: RecordedEvent[];
  maxSize: number;
  addEvent(type: string, data: any, source: string): void;
  detectPatterns(): DetectedPattern[];
}

// Detected patterns inform spirit decisions
interface DetectedPattern {
  type: string;  // "repetition", "conflict", "stagnation", "emergence"
  description: string;
  frequency: number;
  relevantEvents: RecordedEvent[];
}
```

15.3 Context Generation

Spirits can request full system context:

```
function getIntrospectionContext(world: World): IntrospectionContext {
  return {
    entities: getEntitySnapshot(world),
    components: COMPONENT_REGISTRY,
    actions: ACTION_REGISTRY,
    systems: getActiveSystems(),
    recentEvents: getRecentEvents(100),
    detectedPatterns: detectPatterns(),
```

```
    };
  }
```

Appendix: How the Two Architecture Docs Relate

ARCHITECTURE.md	WORLDBUILDING_ARCHITECTURE.md
How agents think	How GodAI builds worlds
Mind, Stimulus, CognitiveEvent	Components, Systems, Rules
Cognitive loop	Simulation tick loop
Individual agent focus	World-building focus
Internal cognition	External world structure

Both are essential: GodAI builds the world (this doc), agents think within it (ARCHITECTURE.md).